Automatic Air Defense System

Arduino-Based Radar-Guided Turret — 3D-Printed Enclosure with OLED Radar Display

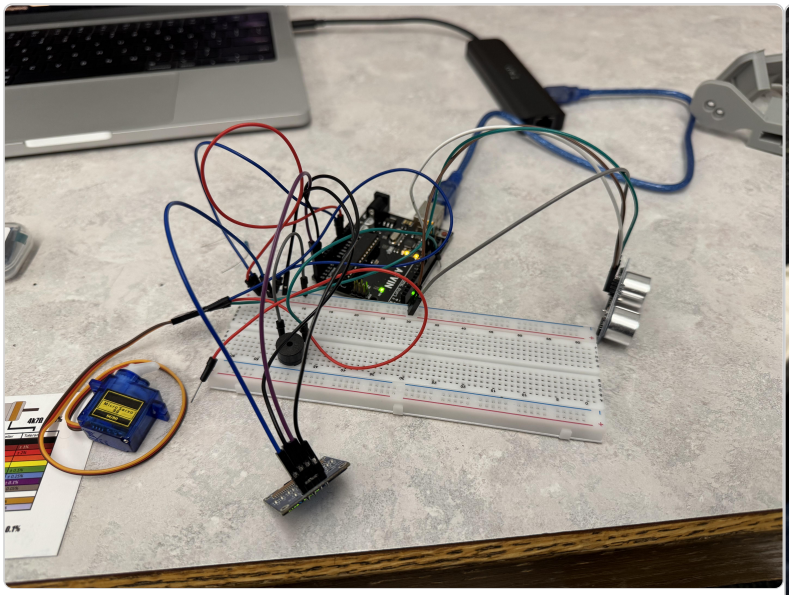
Designer	Marcos Perez
Tools & Skills Used	Arduino (C/C++), SolidWorks / Fusion 360 (3D-printed housing), Servo motor control, Ultrasonic sensing, OLED display programming (U8g2 library), Circuit prototyping (breadboard)
Hardware	Arduino Uno, 4x SG90 micro servos, HC-SR04 ultrasonic distance sensor, SSD1306 OLED display, piezo buzzer, 3D-printed turret housing
Demo Video	https://www.youtube.com/watch?v=GHF34XWE1ww
Reference Project	Automatic Missile Defence System (Instructables) — used as a design reference for the turret/firing mechanism

Project Overview

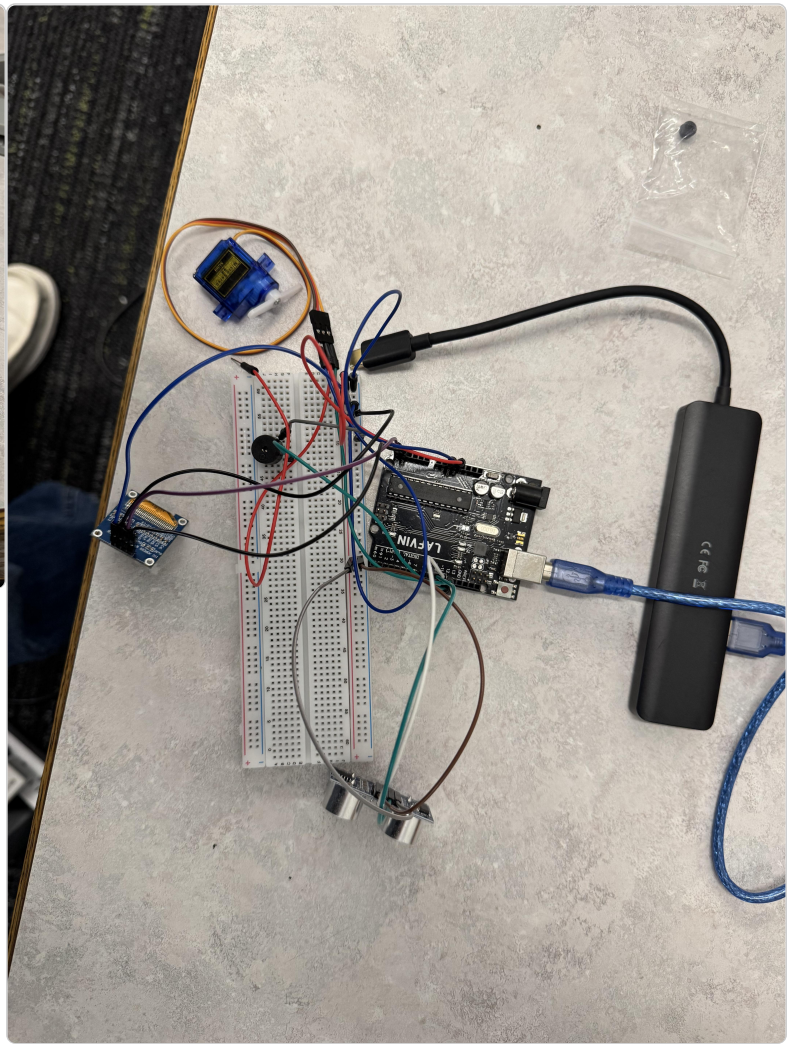
This project is a radar-guided automatic turret built around an Arduino Uno. An ultrasonic sensor is mounted on a servo-driven sweeping arm, continuously scanning the area in front of it. As it sweeps, an OLED screen renders a live radar display — showing the current scan angle and a dot for any object detected within range, similar to a classic sonar/radar sweep. A piezo buzzer provides audio feedback: a soft tick on every scan step, and a rising-pitch alert tone as an object gets closer.

When an object is detected within the defined firing range, the system automatically triggers a launch action using a dedicated trigger servo, mounted inside a 3D-printed turret housing alongside the pan and tilt servos that aim the mechanism.

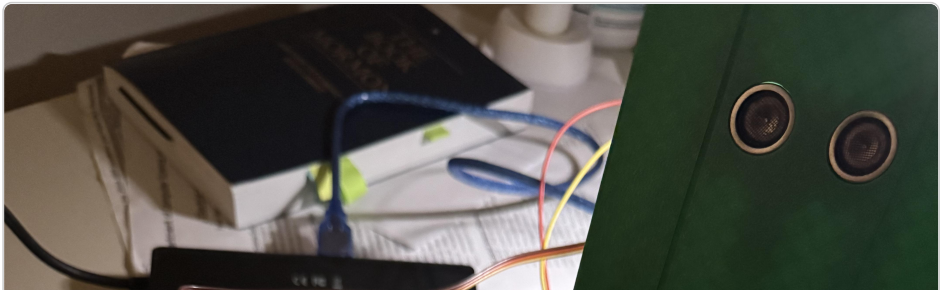
Build Photos

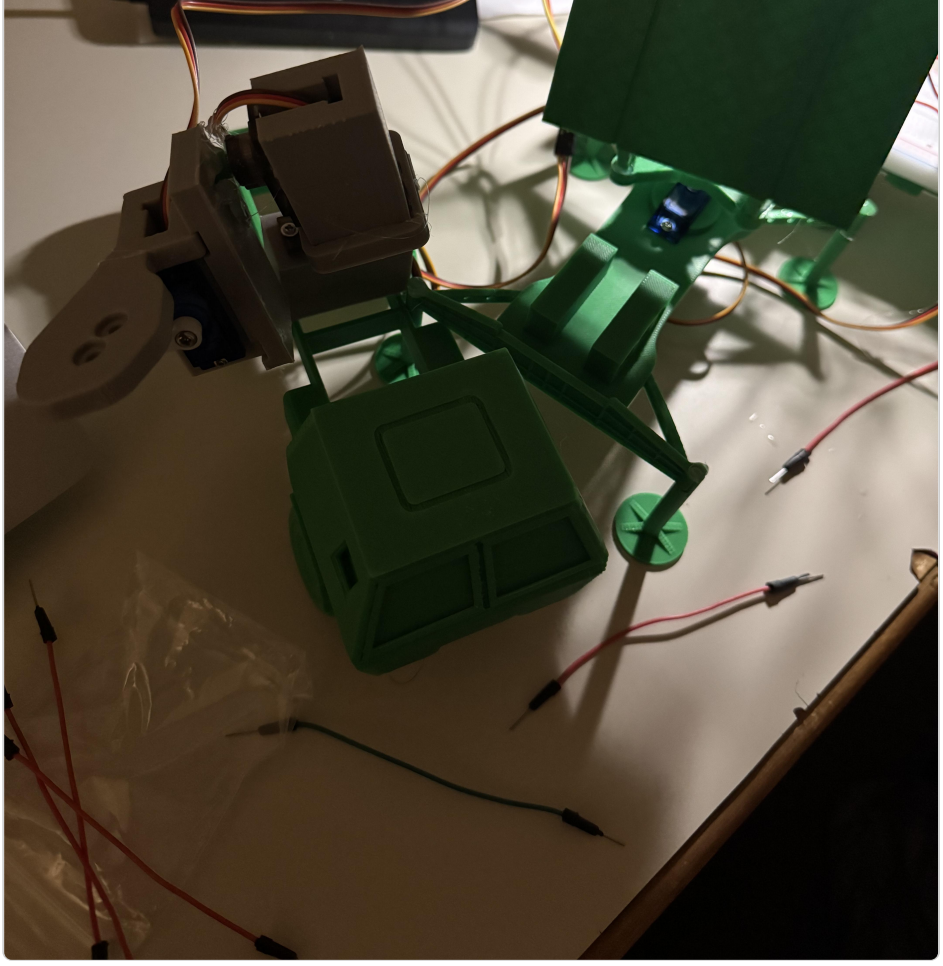


Breadboard prototype: Arduino, ultrasonic sensor, OLED display, buzzer, and servo wiring.



Top-down view of the wired circuit, showing sensor and servo connections to the Arduino.





3D-printed turret housing with pan/tilt servo assembly and launcher body mounted.

How It Works

- **Scanning:** A pan servo sweeps the ultrasonic sensor from 15° to 165° and back, continuously measuring distance at each step.
- **Radar Display:** The OLED screen draws a rotating sweep line synced to the servo's current angle, along with a dot marking any detected object's position and distance — recreating a classic radar sweep visual.
- **Audio Feedback:** A buzzer ticks once per scan step, and plays a rising-pitch alert tone as an object gets closer, giving an audible sense of proximity.
- **Automatic Firing:** When an object is detected within the defined firing range, the trigger servo fires the launch mechanism, then resets to its rest position before the scan continues.

> **Development note:** The radar display/audio system and the turret firing mechanism were originally built and tested as two separate Arduino sketches. The firing/turret code was adapted using the referenced Instructables project as a starting point for the multi-servo turret logic, then combined with the radar/OLED code into a single working sketch (included below) so that detection and firing happen together in real time, driven by one shared sensor sweep instead of two independent systems.

Code

1. Radar Display & Sound System (original sketch)

Handles the OLED radar sweep visualization and buzzer feedback based on live ultrasonic readings.

```
#include <Servo.h>
#include <U8g2lib.h>
#include <Wire.h>

const int trigPin = 10;
const int echoPin = 11;
const int buzzerPin = 6;
Servo myServo;
long duration;
int distance;
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, U8X8_PIN_NONE);

void setup() {
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  Serial.begin(9600);
  myServo.attach(12);
  u8g2.begin();
  u8g2.setContrast(100);
  playStartupSound();
}

void loop() {
  for (int angle = 15; angle <= 165; angle += 2) scanAndDraw(angle);
  for (int angle = 165; angle >= 15; angle -= 2) scanAndDraw(angle);
```

```
}
// ... full sweep, sensor, display, and sound functions
// (see radar_project.ino for complete listing)
```

2. Turret & Firing Mechanism (original sketch, adapted from reference project)

Controls the 3-servo turret (base/pan, middle/tilt, top/trigger) and fires when a target is within range.

```
#include <Servo.h>
#define trigPin 2
#define echoPin 3
Servo servo_base, servo_middle, servo_top;
long duration; int distance;

void setup() {
  servo_base.attach(9);
  servo_middle.attach(10);
  servo_top.attach(5);
  servo_top.write(80);
  servo_middle.write(90);
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

void loop() {
  for (int i = 0; i <= 180; i += 5) {
    ultrasonic();
    servo_base.write(i);
    // tilt table sets servo_middle position based on i ...
    while (distance <= 30) {
      servo_top.write(40);   // fire
      delay(500);
      ultrasonic();
    }
    servo_top.write(80);     // reset
  }
  // mirrored sweep back from 180 to 0 ...
}
// (see final_code_missile_Launcher_22.ino for complete listing)
```

3. Combined System (final, integrated sketch)

Merges both systems above into one program: a single sweep drives the radar display, sound, and automatic firing together.

```
#include <Servo.h>
#include <U8g2lib.h>
#include <Wire.h>

const int trigPin = 10, echoPin = 11, buzzerPin = 6;
const int panServoPin = 12, tiltServoPin = 4, fireServoPin = 5;
const int FIRE_RANGE_CM = 30, FIRE_ANGLE = 40, REST_ANGLE = 80;

Servo panServo, tiltServo, fireServo;
long duration; int distance;
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, U8X8_PIN_NONE);

void setup() {
  pinMode(trigPin, OUTPUT); pinMode(echoPin, INPUT); pinMode(buzzerPin, OUTPUT);
  Serial.begin(9600);
  panServo.attach(panServoPin);
  tiltServo.attach(tiltServoPin);
  fireServo.attach(fireServoPin);
  fireServo.write(REST_ANGLE);
  tiltServo.write(90);
  u8g2.begin();
  u8g2.setContrast(100);
  playStartupSound();
}

void loop() {
  for (int angle = 15; angle <= 165; angle += 2) scanAndRespond(angle);
  for (int angle = 165; angle >= 15; angle -= 2) scanAndRespond(angle);
}

void scanAndRespond(int angle) {
  panServo.write(angle);
  tiltServo.write(map(angle, 15, 165, 70, 110));
  delay(15);
  distance = calculateDistance();
  radarBeep();
  drawRadar(angle, distance);
  if (distance > 0 && distance < FIRE_RANGE_CM) {
    playDetectionAlert(distance);
    fire();
  }
}

void fire() {
  fireServo.write(FIRE_ANGLE);
  delay(500);
  fireServo.write(REST_ANGLE);
}
// calculateDistance(), drawRadar(), playStartupSound(),
// radarBeep(), and playDetectionAlert() carried over from the radar sketch above.
```